

Static semantics of programming languages

Overview

- ▶ defined only for **statically typed languages**
- ▶ implemented by a **typechecker**

Motivations

- ▶ parsers check only syntactic errors
- ▶ typecheckers can detect bugs that manifest at runtime

Recap

- ▶ **static** means “before execution”
- ▶ **dynamic** means “during the execution” (that is, “at runtime”)

Static semantics of programming languages

Syntax versus static semantics

- ▶ **theoretical limitation**: static semantics cannot be specified with regular expressions and CF grammars
- ▶ **practical issue**: the implementation is simpler and the code better organized if checks are separated:
 - phase 1**: syntax checks and AST generation (parser)
 - phase 2**: type checks (typechecker)

Recall: AST = Abstract Syntax Tree

Static semantics of programming languages

Main ingredients

- ▶ **static types** (or simply types) specify sets of values
example: `int` is the set of all integer values
- ▶ **typing rules**:
 - ▶ specify the expressions and statements which are **statically correct** and those which are **not**
 - ▶ specify the type of each correct expression
 - ▶ remark: statements do not have types

Example of type errors

- ▶ illegal operations
- ▶ illegal memory access

Simple examples

Statically correct

<code>1*-2+1</code>	has type <code>int</code>
<code>fst (true, 2)</code>	has type <code>bool</code>
<code>if(2==1+1) {print 1}</code>	is statically correct

Not statically correct

<code>1*true</code>	<u>error</u> : found <code>bool</code> , expected <code>int</code>
<code>fst 2</code>	<u>error</u> : found <code>int</code> , expected <code>pair</code>
<code>if(2) {print 1}</code>	<u>error</u> : found <code>int</code> , expected <code>bool</code>

Remark: `fst` is an operator on pairs

More complex examples with variables

Possibly statically correct expressions

$1 * x$ has type `int` if x is defined and has type `int`

`fst y` has type t_1 if y is defined and has type $t_1 * t_2$

Remark: $t_1 * t_2$ is the type of pairs with values of type t_1 and t_2 , respectively

Correct programs

```
var z=true;
if(z) {print 1}
```

is statically correct

```
var z=true;
if(z) {var z=1; print z * 2}
```

is statically correct

More complex examples with variables

Programs which are not statically correct

```
if(z) {print 1}
```

error: undeclared variable `z`

```
var z=1;
```

```
if(z) {print 1}
```

error: found `int`, expected `bool`

```
var z=false;
```

```
if(z) {var z=1; print true && z}
```

error: found `int`, expected `bool`

Variable declarations

Declarations can be nested

example:

```
var z=true; // declaration at top-level 0  
  if(z) {  
    var z=1; // nested declaration at level 1  
    print z * 2  
  }
```

Variable declarations

Static environments track variable declarations

a static environment defines the information associated with the variables defined at a specific point in the program:

- ▶ the defined **variables**
- ▶ the **types** associated with them
- ▶ the **scope level** of their declarations

Scope of a variable declaration

the **scope of a variable declaration** is **the code fragment where the programmer can correctly use that declared variable**

Variable declarations

Standard typing rules on variables declarations

- ▶ variables must be declared **before** their use
- ▶ declarations can be **nested** by using **blocks**
- ▶ variables with the **same name** cannot be **re-declared** in the **same scope level**
- ▶ declarations in **nested scope levels shadow** the declarations of variables with the same name in **outer enclosing scope levels**

Nested declarations

Example with nested declarations

```

1  1  var x=1;
2  2  if(x==1) {
3  3      var x=true; // nested, outer declaration of x is shadowed
4  4      print x && true // statically correct
5  5  };
6  6  print x + 1          // statically correct

```

Remarks:

- ▶ the scope of `var x=1` coincides with lines 2, 3 and 6
remark: on line 3 the scope is restricted to the right-hand side of the declaration
- ▶ the scope of `var x=true` coincides with line 4
- ▶ `var x=1` is **shadowed** by `var x=true` in line 4

Definition of the semantics of a programming language

Possible approaches

1. mathematical language

- ▶ pros: very abstract, non ambiguous, concise, use of proof assistants
- ▶ cons: based on complex concepts, not directly executable, not suitable for some details

2. declarative programming language (functional or logic)

- ▶ pros: abstract, non ambiguous, concise, executable
- ▶ cons: requires knowledge of the used programming language

3. natural language

- ▶ pros: intuitive, requires minimal technical skills to be understood
- ▶ cons: ambiguous, verbose, not executable, not suitable for technical details

Definition of the static semantics

Our choice

we follow the first and second approach, by defining the semantics of our toy language both in a mathematical way and by using F#

Remarks

although executable, the F# program is only used as a formal definition but not as an implementation of the language, because

- ▶ it does not provide a parser
- ▶ it is not meant to be efficient

How can we model environments?

A standard approach

- ▶ **static environment** = a non-empty sequence of scope levels
 $[m_{n-1}, \dots, m_0]$
- ▶ **scope level** = a finite partial map $m : \text{Variable} \rightarrow \text{StaticType}$ from variables to types
 - ▶ *partial*: not defined for all variables, but only for those declared at that level
 - ▶ *finite*: only a finite number of variables can be declared at a level
 - ▶ $\text{dom}(m)$: the domain of m is the set of variables declared in m

How can we model environments?

A standard approach

- ▶ scope levels are **sorted**: **inner** levels come **first** because **nested** declarations **shadow** declarations in **outer enclosing levels**
- ▶ **remark**: the **first level** in the sequence corresponds to the **current level**, which is the **innermost**

an environment with n scope levels

$[m_{n-1}, m_{n-2}, \dots, m_0]$

current and innermost scope level

top and outermost scope level

Static semantics: formalization

The environment Γ is a sequence of maps associating variables with static types τ

Types:

$$\tau ::= \text{int} \mid \text{bool} \mid \tau \times \tau$$

Remark: the type constructor $\times$ is used to define the type of pairs. For example, the type of pairs with an integer and a boolean component is $\text{int} \times \text{bool}$

The typing relation for expressions is denoted by:

$$\Gamma \vdash e : \tau$$

which means that expression e has type τ in the environment Γ

Typing rules: expressions

T-INT

$$\frac{}{\Gamma \vdash n : \text{int}}$$

T-BOOL

$$\frac{}{\Gamma \vdash b : \text{bool}}$$

T-VAR

$$\frac{\text{lookup}(x, \Gamma) = \tau}{\Gamma \vdash x : \tau}$$

T-ADD

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 + e_2 : \text{int}}$$

T-MUL

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash e_1 * e_2 : \text{int}}$$

T-MINUS

$$\frac{\Gamma \vdash e : \text{int}}{\Gamma \vdash -e : \text{int}}$$

Typing rules: expressions (cont.)

$$\text{T-EQ} \quad \frac{\Gamma \vdash e_1 : \tau \quad \Gamma \vdash e_2 : \tau}{\Gamma \vdash e_1 == e_2 : \text{bool}}$$

$$\text{T-AND} \quad \frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \text{bool}}{\Gamma \vdash e_1 \&\& e_2 : \text{bool}}$$

$$\text{T-NOT} \quad \frac{\Gamma \vdash e : \text{bool}}{\Gamma \vdash !e : \text{bool}}$$

$$\text{T-PAIR} \quad \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1, e_2) : \tau_1 \times \tau_2}$$

$$\text{T-FST} \quad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{fst } e : \tau_1}$$

$$\text{T-SND} \quad \frac{\Gamma \vdash e : \tau_1 \times \tau_2}{\Gamma \vdash \text{snd } e : \tau_2}$$

Derivation trees

- ▶ derivations trees are built by applying the typing rules
- ▶ derivation for the expression $(2 == x) \ \&\& \ (fst(y, true) == 2)$
- ▶ environment $\Gamma = \{y \mapsto int\} :: \{x \mapsto int, y \mapsto bool\} :: []$

$$\frac{
 \frac{
 \frac{}{\Gamma \vdash 2 : Int} \text{ T-Int} \quad
 \frac{
 \frac{lookup(x, \Gamma) = Int}{\Gamma \vdash x : Int} \text{ T-Var}
 }{\Gamma \vdash 2 == x : Bool} \text{ T-Eq}
 }{\Gamma \vdash (2 == x) \ \&\& \ (fst(y, true) == 2) : Bool} \text{ T-And}
 }{
 \frac{
 \frac{
 \frac{lookup(y, \Gamma) = Int}{\Gamma \vdash y : Int} \text{ T-Var} \quad
 \frac{
 \frac{}{\Gamma \vdash true : Bool} \text{ T-Bool}
 }{\Gamma \vdash (y, true) : Int \times Bool} \text{ T-Pair}
 }{\Gamma \vdash fst(y, true) : Int} \text{ T-Fst}
 }{\Gamma \vdash fst(y, true) == 2 : Bool} \text{ T-Eq}
 }{\Gamma \vdash (2 == x) \ \&\& \ (fst(y, true) == 2) : Bool} \text{ T-And}
 }$$

Remarks:

- ▶ y is resolved to `int` because it is found in the innermost level
- ▶ x is found at the top level

Derivation trees: example of type error

- ▶ analysis of the ill-typed expression `true == y`
- ▶ environment $\Gamma = \{y \mapsto \text{int}\} :: \{x \mapsto \text{int}, y \mapsto \text{bool}\} :: []$

$$\frac{\frac{}{\Gamma \vdash \text{true} : \text{Bool}} \text{ T-Bool} \quad \frac{\text{lookup}(y, \Gamma) = \text{Int}}{\Gamma \vdash y : \text{Int}} \text{ T-Var}}{\Gamma \vdash \text{true} == y : ???} \text{ T-Eq (FAIL)}$$

- ▶ **type mismatch**: the rule T-Eq for equality requires both operands to have the same type τ
- ▶ **result**: no derivation tree can be built, therefore the expression is **rejected** by the static semantics
- ▶ **possible error message**:
type mismatch: found **int**, expected **bool**

Typing rules: statements

The relation $\Gamma \vdash s : \Gamma'$ means that statement s is statically correct in the environment Γ and that the resulting environment after executing s is Γ'

$$\frac{\text{S-ASSIGN} \quad \text{lookup}(x, \Gamma) = \tau \quad \Gamma \vdash e : \tau}{\Gamma \vdash x = e : \Gamma}$$

$$\frac{\text{S-VAR} \quad x \notin \text{dom}(m) \quad m :: \Gamma \vdash e : \tau}{m :: \Gamma \vdash \text{var } x = e : \text{dec}(x, \tau, m :: \Gamma)}$$

$$\frac{\text{S-PRINT} \quad \Gamma \vdash e : \tau}{\Gamma \vdash \text{print } e : \Gamma}$$

$$\frac{\text{S-ASSERT} \quad \Gamma \vdash e : \text{bool}}{\Gamma \vdash \text{assert } e : \Gamma}$$

Typing rules: conditional statement

S-IF-THEN-ELSE

$$\frac{\Gamma \vdash e : \text{bool} \quad \Gamma \vdash \text{block}_{\text{then}} \quad \Gamma \vdash \text{block}_{\text{else}}}{\Gamma \vdash \text{if } (e) \text{ block}_{\text{then}} \text{ else block}_{\text{else}} : \Gamma}$$

Typing rules: statement sequences and blocks

The relation $\Gamma \vdash \bar{s}$ means that the sequence of statements $\bar{s}$ is statically correct in the environment Γ

SSEQ-EMPTY

$$\frac{}{\Gamma \vdash \epsilon}$$

SSEQ-NONEMPTY

$$\frac{\Gamma \vdash s : \Gamma' \quad \Gamma' \vdash \bar{s}}{\Gamma \vdash s; \bar{s}}$$

The relation $\Gamma \vdash \{\bar{s}\}$ means that the block $\{\bar{s}\}$ is statically correct in the environment Γ

B-BLOCK

$$\frac{\{ \} :: \Gamma \vdash \bar{s}}{\Gamma \vdash \{\bar{s}\}}$$

Static semantics: programs

The relation $\vdash \bar{s}$ means that the program $\bar{s}$ is statically correct in the initial environment

$$\frac{\text{P-PROG} \quad \{ \} :: [] \vdash \bar{s}}{\vdash \bar{s}}$$

- ▶ $\{ \} :: []$ is the list with a single empty map $\{ \}$
- ▶ this map represents the initially empty set of top-level declarations
- ▶ $[]$ is the empty list of maps: no nested levels are present at the beginning of the program

Static semantics: auxiliary functions on environments

$\text{lookup} : \text{Variable} \times \text{StaticEnv} \rightarrow \text{StaticType}$

$\text{lookup}(x, \Gamma)$ checks if variable x is declared in Γ and returns its type

$$\text{lookup}(x, m :: \Gamma) = \begin{cases} m(x) & \text{if } x \in \text{dom}(m) \\ \text{lookup}(x, \Gamma) & \text{if } x \notin \text{dom}(m) \end{cases}$$

$\text{lookup}(x, []) \text{ is undefined}$

recall:

a map $m : \text{Variable} \rightarrow \text{StaticType}$ is a finite partial map: if x is not declared in m , then it is not in the domain of m ($x \notin \text{dom}(m)$), therefore $m(x)$ is undefined

Static semantics: auxiliary functions on environments

- ▶ $\text{dec} : \text{Variable} \times \text{StaticType} \times \text{StaticEnv} \rightarrow \text{StaticEnv}$ is a partial function which models the declaration of a new variable in the current environment
- ▶ $\text{dec}(x, \tau, \Gamma)$ returns the environment obtained by declaring the new variable x with type τ at the current scope level of Γ

$$\text{dec}(x, \tau, m :: \Gamma) = (m \cup \{x \mapsto \tau\}) :: \Gamma \text{ if } x \notin \text{dom}(m)$$

$$\text{dec}(x, \tau, m :: \Gamma) \text{ is undefined if } x \in \text{dom}(m)$$

$$\text{dec}(x, \tau, []) \text{ is undefined}$$

Static environments in F#

Technical details

- ▶ definition of static type

```
type staticType = | IntType | BoolType | PairType of staticType *  
                  staticType
```

- ▶ definition of static environment

```
type variable = Name of string  
type scopeLevel = Map<variable, staticType>  
type staticEnv = scopeLevel list
```

- ▶ some basic environments

- ▶ a level with no declarations

```
let emptyLevel: scopeLevel = Map.empty
```

- ▶ the initial environment before the program will be executed

```
let initialEnv: staticEnv = [ emptyLevel ]
```

Static environments in F#

Technical details

dictionaries are implemented by the `Map` module

`Map.empty`: `Map<'a, 'b>` when `'a`: `comparison`

`Map.empty` returns the empty dictionary

`Map.add`: `('a -> 'b -> Map<'a, 'b> -> Map<'a, 'b>)` when `'a`:
`comparison`

`Map.add k v d` returns a new dictionary where the key `k` is associated with `v` in `d`

`Map.tryFind`: `('a -> Map<'a, 'b> -> 'b option)` when `'a`: `comparison`

`Map.tryFind k d` returns `Some v` if `k` is associated with `v` in `d`, `None` otherwise

`Map.containsKey`: `('a -> Map<'a, 'b> -> bool)` when `'a`: `comparison`

`Map.containsKey k d` checks if the key `k` is contained in the dictionary `d`

Static environments in F#

Example with two nested scope levels

a list of two scope levels

```
env = [map [(Name "x", IntType); (Name "y", BoolType)]; map [(Name "x", BoolType); (Name "z", IntType)]]
```

declarations at level 1: x of type int and y of type bool

declarations at level 0 (=top-level): x of type bool and z of type int

Static environments in F#

Functions for handling the static environment

`enterLevel: staticEnv -> staticEnv`

adds a nested scope level because a block `{ ... }` is entered

`exitLevel: staticEnv -> staticEnv`

removes the current scope level because a block `{ ... }` is exited

`lookup: variable -> staticEnv -> staticType`

searches the definition of a variable

`dec: variable -> staticType -> staticEnv -> staticEnv`

declares a new variable at the current level; raises an exception if the variable has been already declared in the current level

Static environments: function lookup

Definition

```
lookup: variable -> staticEnv -> staticType
```

- ▶ function `lookup` searches the definition of a variable
- ▶ the search starts from the most nested level and continues in the outer levels until the variable is found or no levels are left
- ▶ if the variable is found, then its associated type is returned
- ▶ otherwise an exception is raised

Examples of variable lookup

let us consider the environment `env` as defined on page **28**

```
assert ((lookup (Name "x") env) = IntType)
assert ((lookup (Name "y") env) = BoolType)
assert ((lookup (Name "z") env) = IntType)
```

```
lookup (Name "w") env raises exception UndeclaredVariable (Name "w")
```

Abstract syntax of the language

```

type exp =
  | Add of exp * exp           // addition
  | Mul of exp * exp           // multiplication
  | And of exp * exp           // logical AND
  | Eq of exp * exp            // equality
  | PairLiteral of exp * exp   // pair constructor
  | Fst of exp                 // first element of a pair
  | Snd of exp                 // second element of a pair
  | Minus of exp               // unary subtraction
  | Not of exp                 // logical NOT
  | IntLiteral of int          // integer literal
  | BoolLiteral of bool        // boolean literal
  | Var of variable            // variable

type stmt =
  | AssignStmt of variable * exp // assignment
  | VarStmt of variable * exp    // declaration statement
  | PrintStmt of exp              // print statement
  | IfStmt of exp * block * block // if-then-else
  | AssertStmt of exp             // assert statement

and block =
  | Block of stmtSeq // empty or non-empty block

and stmtSeq =
  | EmptyStmtSeq // empty statement sequence
  | NonEmptyStmtSeq of stmt * stmtSeq // non-empty statement sequence

type prog = ExpProg of stmtSeq // program

```

Definition of the static semantics in F#

Typing functions

remark: in case of **type errors**, an exception is raised

`typecheckProg: prog -> unit`

checks that a program is statically correct starting from `initialEnv`
(see page 26)

`typecheckStmtSeq: staticEnv -> stmtSeq -> unit`

checks that a sequence of statements is statically correct in a static environment

`typecheckBlock: staticEnv -> block -> unit`

checks that a block of statements is statically correct in a static environment

Definition of the static semantics in F#

Typing functions

```
typecheckStmt: staticEnv -> stmt -> staticEnv
```

checks that a statement s is statically correct in a static environment and returns the environment produced by the execution of s

remark: the returned environment tracks the newly declared variable, if any

```
typecheckExp: staticEnv -> exp -> staticType
```

checks that an expression is statically correct in a static environment and returns its static type

Definition of the static semantics

The definition of the typing functions is driven by the typing rules

Functions recursively defined on the AST

- ▶ a case for each type of node of the AST
- ▶ typechecking of a program = a visit of its AST

Typing rules versus F# typing functions

The definition of the static semantics is less abstract in F#

- ▶ the typing rules do **not** specify the type of detected errors when no derivation tree can be built in case of errors
- ▶ in F# different types of exceptions can be raised
(`UndeclaredVariable`, `AlreadyDeclaredVariable`,
`ExpectingTypeError`, ...)
- ▶ no order is defined in the hypotheses of the typing rules
- ▶ in F# the program is analyzed in the deterministic left-to-right order

Implementation of a selection of the typing rules

Variable

expression Variable var



if `lookup var env = ty`

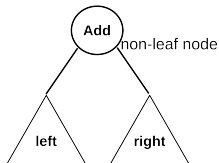
then Variable var has type `ty` in the environment `env`

else it is **not** statically correct in `env` and a static error is raised

Implementation of a selection of the typing rules

Addition

expression `Add(left, right)`



if `left` and `right` have type `int` in the environment `env`

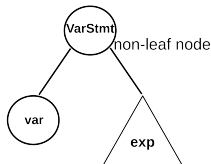
then `Add(left, right)` has type `int` in `env`

else it is **not** statically correct in `env` and a static error is raised

Implementation of a selection of the typing rules

Variable declaration

statement `VarStmt (var, exp)`



if `exp` has type `ty` in the environment `env`,

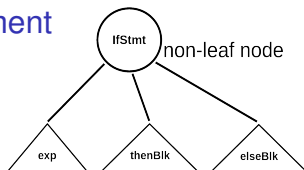
and `dec var ty env = env2`

then `VarStmt (var, exp)` is statically correct in `env` and produces the environment `env2`

else it is **not** statically correct in `env` and a static error is raised

Implementation of a selection of the typing rules

if-then-else statement



if `exp` has type `bool` in the environment `env`,
and `thenBlk` and `elseBlk` are statically correct in `env`

then `IfStmt(exp, thenBlk, elseBlk)` is statically correct in `env`
and returns the same environment `env`

else it is not statically correct in `env` and a static error is raised